

Shooting Snake

Snake Survival: Action-Based Data Analysis Game

1. Project Overview

This project is a real-time action game based on the classic snake concept. The player controls a snake using the mouse to collect food, avoid obstacles, and survive as long as possible. The game introduces additional mechanics such as enemy snakes, shooting, and dynamic difficulty.

The game is designed for general players and focuses on both gameplay experience and data collection for statistical analysis.

2. Project Review

A common reference game is the traditional Snake game, where players control a snake to collect food and grow longer.

Our project improves upon this by:

- Adding enemy snakes with independent movement
- Introducing shooting mechanics
- Including obstacles and dynamic difficulty
- Collecting detailed gameplay statistics for analysis

These enhancements make the game more engaging and suitable for data-driven evaluation.

3. Programming Development

3.1 Game Concept

The player controls a snake that moves toward the mouse cursor. The goal is to survive for 2 minutes while collecting food, avoiding obstacles, and defeating enemy snakes.

When the player collides with an enemy or obstacle, the game does not restart. Instead, the player receives a score penalty, reducing the current score while the game continues. This allows continuous gameplay and smoother data collection.

Key features:

- Mouse controlled movement
- Food collection and growth system
- Enemy snakes with random movement
- Shooting system to eliminate enemies
- Obstacles that increase difficulty
- Score penalty system instead of full reset on death
- Continuous 2-minute gameplay session

3.2 Object Oriented Programming Implementation

The game will be implemented using at least 5 classes:

1. Game
 - Handles the main game loop, timing system, event handling, and data recording.
2. Snake (Player)
 - Attributes: position, body, length
 - Methods: move(), grow(), check_collision()
3. EnemySnake
 - Inherits from Snake
 - Moves randomly and acts as an enemy entity
4. Food
 - Spawns randomly in the game environment
 - Increases player score and snake length when collected
5. Obstacle
 - Static objects placed in the map
 - Cause player death and trigger a score penalty upon collision

3.3 Algorithms Involved

- Vector-based movement for mouse tracking
- Collision detection using distance calculation
- Randomized movement for enemy AI
- Event-driven input handling for shooting
- Dynamic difficulty scaling (enemy and obstacle spawning over time)

4. Statistical Data (Prop Stats)

4.1 Data Features

The game collects continuous gameplay data at regular time intervals. The following features are recorded:

1. **Time:** Elapsed time in seconds (0–120)
2. **Score:** Continuous score value, including increases from actions and decreases from death penalties
3. **Food Collected:** Total number of food items collected (cumulative)
4. **Shooting Count:** Total number of shots fired (cumulative)
5. **Kill Count:** Total number of enemies defeated (cumulative)
6. **Hit Accuracy:** Ratio of successful hits to total shots (kills / shooting count)
7. **Death Count:** Total number of deaths (collisions with enemies or obstacles). Each death reduces the score but does not reset the game

Each feature will contain 120 records collected during gameplay.

Feature	Why is it useful?	How will you obtain 100 values?	Which variable / class?	How to display?
Time	Used as reference to track changes over the game	Record every second for 120 seconds	Game (timer)	Line graph (Time vs Score)
Score	Main performance indicator of the player	Record every second (continuous value)	Game	Line graph + statistics table
Food Collected	Shows how	Increase value	Snake / Food	Bar graph +

	well player collects resources	whenever food is collected		statistics
Shooting Count	Shows player activity and aggressiveness	Increase every time player shoots	Game (input system)	Line graph
Kill Count	Shows combat performance	Increase when enemy is defeated	EnemySnake	Bar graph
Hit Accuracy	Shows shooting efficiency	Calculate kills / shooting count per second	Game	Histogram + statistics
Death Count	Shows mistakes and penalties	Increase when player collides with enemy/obstacle	Game	Bar graph + statistics

*Statistical data table

4.2 Data Recording Method

- Data is recorded every second during gameplay
- Each row represents a snapshot of the current game state
- Data is stored in CSV format
- A total of 120 records will be collected per gameplay session

Score is maintained as a continuous variable throughout the session, including reductions caused by death penalties. This approach ensures smoother data trends and improves the reliability of statistical analysis.

4.3 Data Analysis Report

1. Descriptive Statistics

A table will be created to show basic values such as:

- Minimum
- Maximum
- Mean
- Median
- Mode

This helps summarize the data.

Feature	Min	Max	Mean	Median	Mode
Score	N/A	N/A	N/A	N/A	N/A
Food Collected	N/A	N/A	N/A	N/A	N/A
Shooting Count	N/A	N/A	N/A	N/A	N/A
Kill Count	N/A	N/A	N/A	N/A	N/A
Hit Accuracy	N/A	N/A	N/A	N/A	N/A
Death Count	N/A	N/A	N/A	N/A	N/A

*Example of stats table

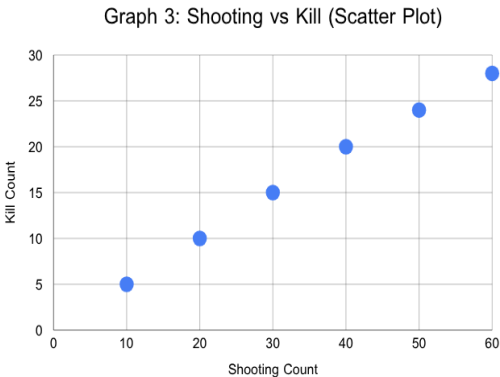
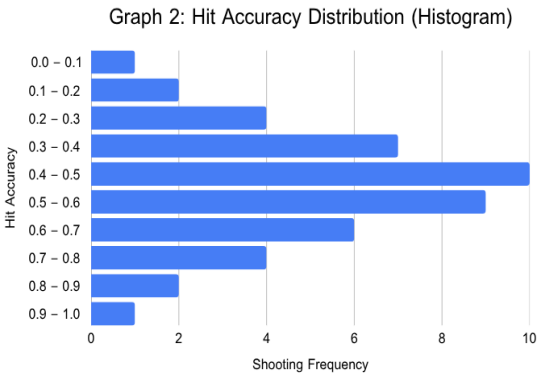
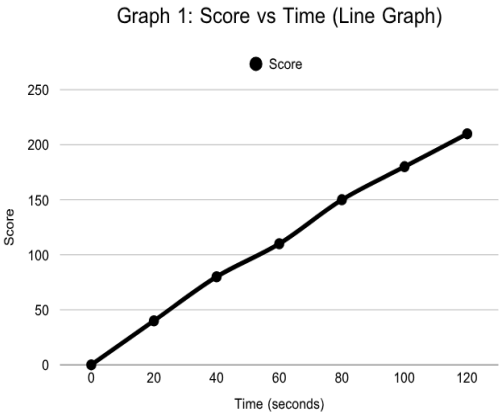
2. Graph Analysis

Graphs will be used to visualize the data:

- **Score vs Time** (Line graph) → shows how the score changes over time
- **Hit Accuracy Distribution** (Histogram) → shows the distribution of player accuracy
- **Shooting Count vs Kill Count** (Scatter plot) → shows the relationship between shooting and kills

Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Score	Show performance over time	Line graph	Time	Score
Hit Accuracy	Show distribution of accuracy	Histogram	Accuracy	Frequency
Shooting vs Kill	Show relationship	Scatter plot	Shooting Count	Kill Count

*graph table



*Graph example

3. Summary

From the data, we can understand:

- How the player gains score over time
- How shooting affects kill count
- How deaths reduce the score due to penalties

The results can be used to improve the game and the analysis will help understand player performance and behavior patterns.

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Game core development
3	10	Add features (enemy, food, obstacle)
4	11	Implement OOP structure
5	12	Data collection system
6	13	Data analysis + visualization
7	14	Final submission

6. Document version

Version: 3.0

Date: 6 April 2026

Editor: Archan Norsiri

Date	Name	feedback
17/03	Peraya	Major Changes Needed - Section 4.1: You are at extreme risk of failing the "100 records per feature" rule. You stated that "Each row will represent one gameplay session." To get 100 records, you (the developer) or a player would have to play the game 100 full times. This is inefficient. You must redefine your features to be high-frequency events recorded <i>during</i> a single game. - Section 3.3: Your description is too generic. "Event handling" and "Movement update" are basic game engine functions, not specific algorithms. I need to see the logic for your Moving Obstacles. - Missing Class Details (Section 3.2): You listed 5 classes, but the attributes and methods are too shallow. For the Snake Class, how is the body stored? Is it a list of coordinates? For the Obstacle Class, how does it interact with the Game Class? Provide more technical depth to prove OOP design. - Presentation Format (Section 4.3): Your analysis plan is too vague. Simply saying "Bar charts and Line graphs" is insufficient.